BIOMEDICAL MOBILE APPLICATION MANUAL QA TESTING PORTFOLIO

Prepared by: Sanjaya Baniya
Role: QA / Software Testing Engineer
Previous Experience: Biomedical Project Internship
Organization: National Innovation Centre (NIC), Nepal
Testing Focus: Manual Testing | Functional Testing | Mobile App QA | Bug Reporting

---

1. PROFESSIONAL PROFILE

I am a QA Tester with practical experience gained through an internship involving a Biomedical project at the National Innovation Centre (NIC), Nepal.

During my internship, I gained experience in:

- Manual Application Testing
- Functional Testing
- Test Case Preparation
- Bug Identification and Reporting
- Testing Documentation

I also have training in software testing and automation using Python, Selenium, Appium, Pytest, and Page Object Model (POM).

I am interested in healthcare and biomedical application testing, where accuracy, reliability, and proper validation are essential.

2. BIOMEDICAL PROJECT EXPERIENCE

Project Domain: Biomedical
Organization: National Innovation Centre, Nepal
Role: Manual QA Testing Intern

Responsibilities and Experience

Manual Testing: Performed manual testing activities to evaluate application behavior and identify unexpected issues.

Functional Testing: Checked application functionalities against expected behavior.

Test Case Preparation: Prepared test scenarios and test cases to verify application functionality.

Bug Identification and Reporting: Identified issues and documented bugs with relevant details.

Testing Documentation: Maintained testing observations and documented testing activities.

3. SAMPLE MEDICAL MOBILE APP TEST PLAN

Project Type: Medical Mobile Application
Testing Approach: Manual Testing
Platforms: Android and iOS

Testing Objectives

- Verify specimen collection workflows.
- Validate sample registration and data handling.
- Test QR code scanning functionality.
- Verify camera permissions and behavior.
- Evaluate scanning under different lighting conditions.
- Check light intensity reading behavior.
- Identify functional and UI issues.
- Verify application compatibility across devices.

4. FUNCTIONAL TEST CASES

Test ID	Test Scenario	Expected Result
TC01	Launch the application	Application opens successfully
TC02	Login with valid credentials	User logs in successfully
TC03	Login with invalid credentials	Error message appears
TC04	Start specimen collection	Collection workflow starts
TC05	Submit incomplete specimen details	Validation message appears
TC06	Submit valid specimen information	Information is saved
TC07	Scan valid QR code	Correct sample information appears
TC08	Scan invalid QR code	Invalid QR error appears
TC09	Scan duplicate QR code	Duplicate entry is handled
TC10	Deny camera permission	Application handles permission correctly
TC11	Allow camera permission	Camera opens successfully

TC12	Navigate between application screens	Navigation works correctly
TC13	Close and reopen application	Saved information remains available
TC14	Submit specimen information without internet	Application handles connectivity correctly
TC15	Restore internet connection	Application handles synchronization correctly

5. QR CODE SCANNING TESTING

Testing Scenarios

1. Scan a valid QR code.
2. Scan an invalid QR code.
3. Scan a damaged or unreadable QR code.
4. Scan a duplicate QR code.
5. Test QR scanning under low-light conditions.
6. Test QR scanning under normal-light conditions.
7. Test QR scanning under bright-light conditions.
8. Verify camera permission handling.

Expected Results

- Valid QR codes should be recognized correctly.
- Invalid QR codes should display appropriate error messages.
- Duplicate samples should be handled according to application rules.
- Scanning should provide appropriate feedback.
- Camera permission issues should not crash the application.

6. CAMERA FUNCTIONALITY TESTING

Test Scenarios

- Verify camera initialization.
- Verify camera permission handling.
- Check camera preview functionality.
- Verify scanning behavior.
- Check camera behavior after application restart.
- Evaluate camera performance under different lighting conditions.
- Verify appropriate error handling when camera access is unavailable.

7. LIGHT INTENSITY (LUX) TESTING

Testing Conditions

Test ID	Environment	Testing Objective
LT01	Low Light	Evaluate scanning and light detection
LT02	Normal Light	Verify normal operating behavior
LT03	Bright Light	Evaluate glare and scanning behavior
LT04	Changing Light	Observe changes in light readings
LT05	Lux Validation	Compare application readings with a calibrated lux meter

Validation Approach

- Compare application lux readings against a calibrated reference meter.
- Record actual readings under different lighting conditions.
- Check whether readings update when lighting changes.
- Document inaccurate readings or unexpected behavior.

Note: Actual lux measurement testing depends on supported device hardware and application sensor integration.

8. SAMPLE BUG REPORT

Bug ID: BUG-001

Title: QR Scanner Not Responding Under Low-Light Conditions

Module: QR Code Scanner

Severity: To be determined after evaluating the impact.

Steps to Reproduce:

1. Open the mobile application.
2. Navigate to the QR scanning screen.
3. Enter a low-light environment.
4. Attempt to scan a valid QR code.
5. Observe the scanner behavior.

Expected Result:

The application should provide appropriate scanning feedback and handle low-light conditions correctly.

Actual Result:

To be recorded during actual execution.

Status: Sample Bug Report – Not Yet Executed.

9. CROSS-DEVICE COMPATIBILITY TESTING

Available Testing Devices:

- Apple iPhone XS – iOS
- Samsung Galaxy A15 – Android

Testing Focus

- Application installation and launch.
- Screen layout and UI consistency.
- Camera permissions.
- QR scanning functionality.
- Application navigation.
- Unexpected crashes or errors.
- Device-specific behavior.

Note: Sensor availability and lux measurement capability must be verified separately on each device.

10. REGRESSION TESTING

After identifying and fixing bugs, previously tested functionalities should be verified again.

Regression Checklist

- Login functionality.
- Specimen registration.
- Sample information validation.
- QR scanning.
- Camera functionality.
- Application navigation.
- Data saving and synchronization.

11. BUG TRACKING AND DOCUMENTATION

I understand the importance of structured bug reporting and issue tracking.

My sample bug documentation includes:

- Bug ID
- Bug Title
- Module
- Steps to Reproduce
- Expected Result
- Actual Result
- Severity
- Screenshots
- Bug Status

GitHub Issues can be used to track and manage identified issues throughout the testing process.

12. TESTING TOOLS AND TECHNICAL KNOWLEDGE

Manual Testing:

- Functional Testing
- Regression Testing
- Exploratory Testing
- Compatibility Testing
- Test Case Preparation
- Bug Reporting

Automation Training:

- Python
- Selenium WebDriver
- Appium
- Pytest
- Page Object Model (POM)

Documentation and Tracking:

- Microsoft Excel
- GitHub Issues
- Test Execution Reports

13. CONCLUSION

My Biomedical project internship experience, combined with my manual QA testing knowledge and software testing training, provides a foundation for contributing to medical mobile application testing.

I am committed to identifying functional issues, validating application workflows, documenting bugs clearly, and helping improve overall application quality.

I am eager to contribute to your team and continue developing my expertise in healthcare and mobile application QA testing.

Prepared by:
Sanjaya Baniya
QA / Software Testing Engineer